NAME:KRISHNAPRIYA P

REG NO:192511279

Course code:CSA0613

Course title: Design and Analysis of Algorithms

CO2_ASSESSMENT TOOL 3 _ DEBUGGING & OPTIMISATION TASK

Title: Debugging Pair Sum Using Optimized Loop Structure

Aim

To identify the logical error in the insertion sort shifting process, explain why elements are misplaced, debug the algorithm step-by-step, improve code readability, and analyze its time complexity.

Software Used

- Python

Algorithm

- Start.
- Read the array elements.
- Assume the first element is already sorted.
- Select the next element as the **key**.
- Compare the key with elements on its left.
- Shift all elements greater than the key one position to the right.
- Insert the key into its correct position.
- Repeat until all elements are sorted.
- Display the sorted array.
- Stop.

Source Code (Python)

main.py

Share Run

```python
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
arr = [5, 3, 8, 4, 2, 7]
print("Original Array:", arr)
sorted_arr = insertion_sort(arr)
print("Sorted Array:", sorted_arr)
```

OUTPUT

Output

```
Original Array: [5, 3, 8, 4, 2, 7]
Sorted Array: [2, 3, 4, 5, 7, 8]

=== Code Execution Successful ===
```

Analysis

Original Error

- Used arr[j] = arr[j-1].
- Shifted elements in the wrong direction.
- Overwrote existing values.
- Produced duplicate elements and incorrect sorting.

Optimized Approach

- Use arr[j + 1] = arr[j] to shift larger elements one position to the right.
- Insert the key after shifting is complete.
- Use meaningful variable names and proper indentation to improve readability.

Documentation

- The logical error was caused by assigning arr[j] = arr[j-1], which overwrites data instead of shifting elements correctly.
- Replacing it with arr[j+1] = arr[j] preserves all elements while creating space for the key.
- The corrected insertion sort correctly places each element in its sorted position.
- Best Case Time Complexity: $\Theta(n)$
- Average Case Time Complexity: $\Theta(n^2)$
- Worst Case Time Complexity: $\Theta(n^2)$
- Space Complexity: $\Theta(1)$